Chapter 14

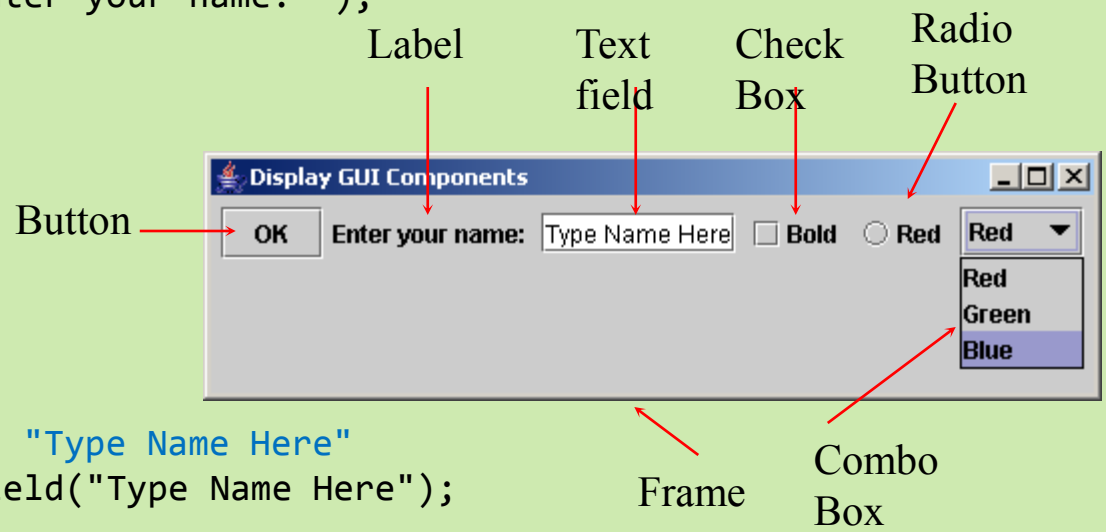
GUI Basics

Objectives

- To distinguish between Swing and AWT (§12.2).
- To describe the Java GUI API hierarchy (§12.3).
- To create user interfaces using frames, panels, and simple GUI components (§12.4).
- To understand the role of layout managers (§12.5).
- To use the FlowLayout, GridLayout, and BorderLayout managers to layout components in a container (§12.5).
- To use JPanel as subcontainers (§12.7).
- To specify colors and fonts using the Color and Font classes (§§12.7-12.8).
- To apply common features such as borders, tool tips, fonts, and colors on Swing components (§12.9).
- To use borders to visually group user-interface components (§12.9).
- To create image icons using the ImageIcon class (§12.10).

Creating GUI Objects

```
// Create a button with text OK
JButton jbtOK = new JButton("OK");

// Create a label with text "Enter your name: "
JLabel jlblName = new JLabel("Enter your name: ");
```

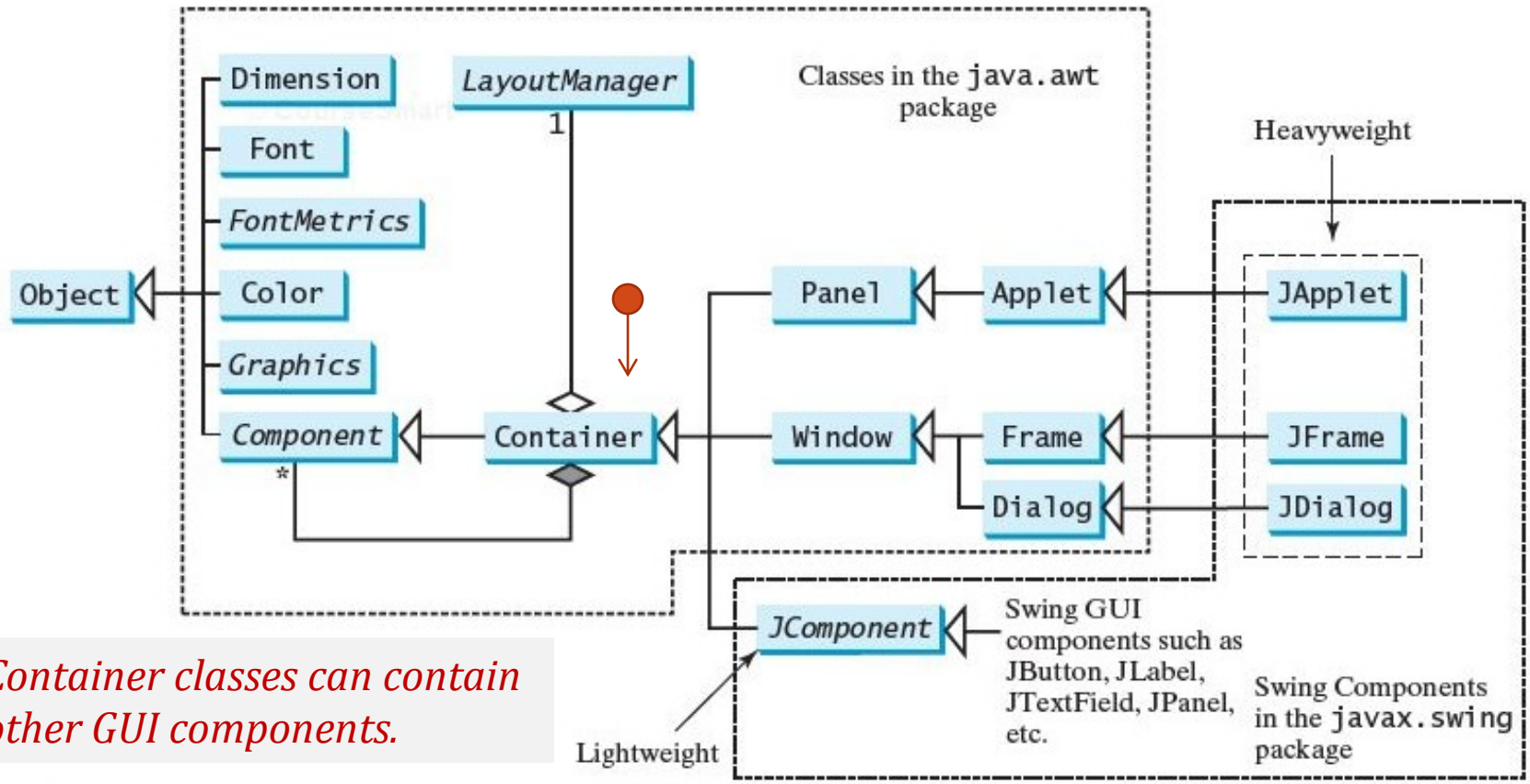



```
// Create a text field with text "Type Name Here"
JTextField jtfName = new JTextField("Type Name Here");

// Create a check box with text bold
JCheckBox jchkBold = new JCheckBox("Bold");

// Create a radio button with text red
JRadioButton jrbRed = new JRadioButton("Red");

// Create a combo box with choices red, green, and blue
JComboBox jcboColor = new JComboBox(new String[]{"Red", "Green", "Blue"});
```

Swing vs. AWT

- First Java GUI library was known as the Abstract Windows Toolkit (AWT).
- AWT is fine for developing simple graphical user interfaces, but not for complex GUI projects.
- A newer, more robust, and flexible library is known as Swing components.
- Swing components are less dependent on the target platform and use less of the native GUI resource.
- Swing components that don't rely on native GUI are referred to as lightweight components and AWT components are referred to as heavyweight components.

Swing - Container Classes



Container classes can contain other GUI components.

FIGURE 12.1 Java GUI programming utilizes the classes shown in this hierarchical diagram.

GUI API - Container Classes

TABLE 12.1 GUI Container Classes

Container Class	*Description*
`java.awt.Container`	is used to group components. Frames, panels, and applets are its subclasses.
`javax.swing.JFrame`	is a window not contained inside another window. It is used to hold other Swing user-interface components in Java GUI applications.
`javax.swing.JPanel`	is an invisible container that holds user-interface components. Panels can be nested. You can place panels inside a container that includes a panel. `JPanel` is also often used as a canvas to draw graphics.
`javax.swing.JApplet`	is a subclass of `Applet`. You must extend `JApplet` to create a Swing-based Java applet.
`javax.swing.JDialog`	is a popup window or message box generally used as a temporary window to receive additional information from the user or to provide notification that an event has occurred.

© CourseSmart

GUI API - Helper Classes

TABLE 12.2 GUI Helper Classes

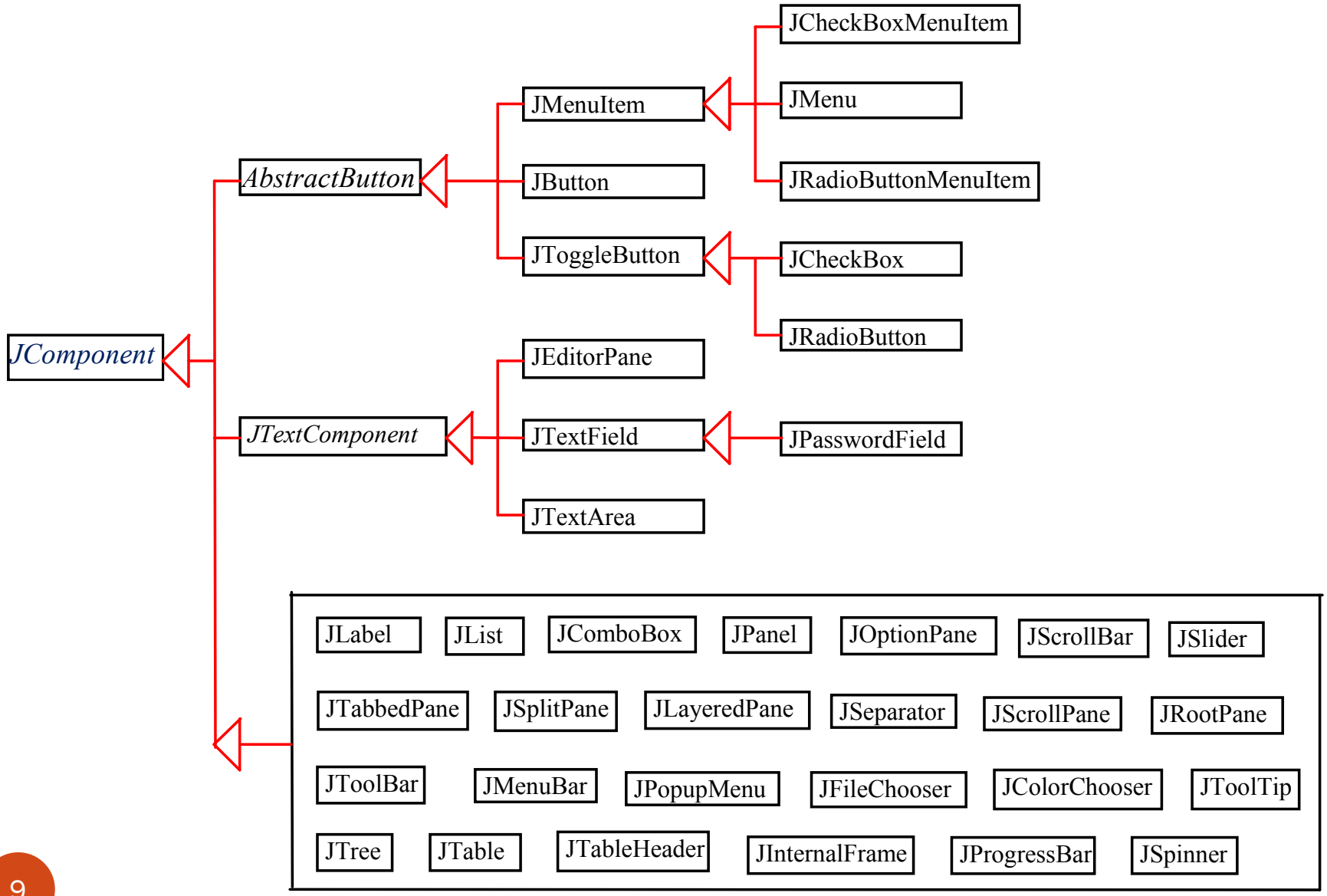
Helper Class	*Description*
`java.awt.Graphics`	is an abstract class that provides the methods for drawing strings, lines, and simple shapes.
`java.awt.Color`	deals with the colors of GUI components. For example, you can specify background or foreground colors in components like `JFrame` and `JPanel`, or you can specify colors of lines, shapes, and strings in drawings.
`java.awt.Font`	specifies fonts for the text and drawings on GUI components. For example, you can specify the font type (e.g., SansSerif), style (e.g., bold), and size (e.g., 24 points) for the text on a button.
`java.awt.FontMetrics`	is an abstract class used to get the properties of the fonts.
`java.awt.Dimension`	encapsulates the width and height of a component (in integer precision) in a single object.
`java.awt.LayoutManager`	specifies how components are arranged in a container.

Use AWT or SWING classes?

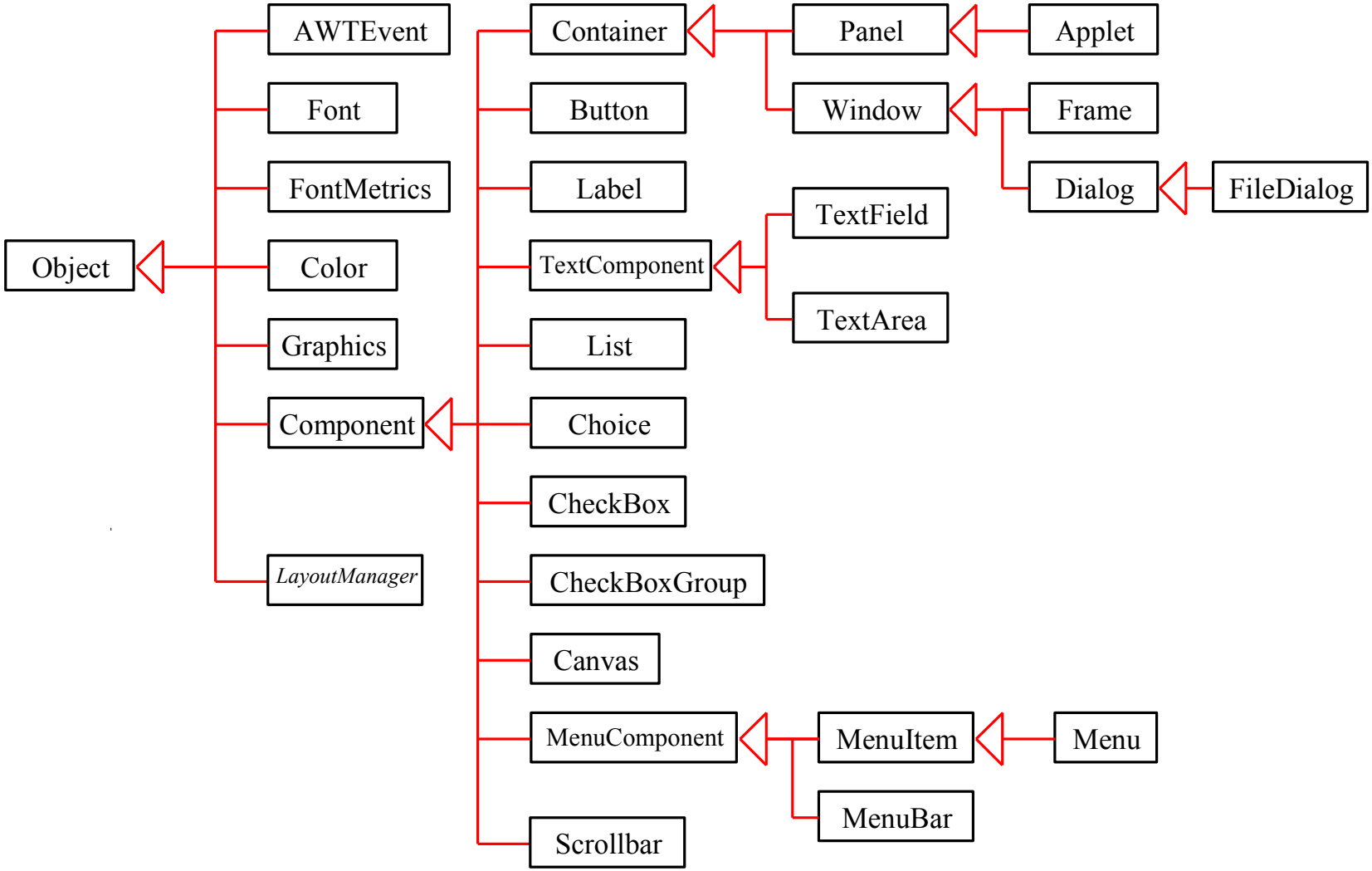
- To distinguish new Swing component classes from their older AWT counterparts, the Swing GUI component classes are named with a prefixed J.

- Although AWT components are still supported in Java, *it is better to learn to how program using Swing components*, because the AWT user- interface components will eventually fade away.

Swing GUI Components



AWT (Optional)



Frames

- To create a user interface, you need to create either a *frame* or an *applet* to hold the user- inter-face components.
- Frame is a window that is not contained inside another window.
- Frame is the basis to contain other user interface components in Java GUI applications.
- The `JFrame` class can be used to create windows.
- For Swing GUI programs, use `JFrame` class to create widows.

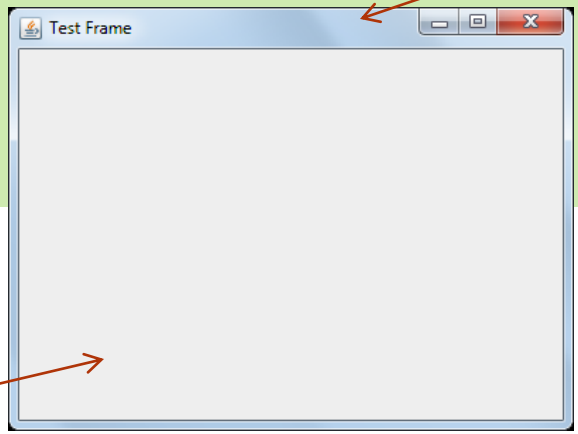
JFrame Class

javax.swing.JFrame	
+JFrame()	Creates a default frame with no title.
+JFrame(title: String)	Creates a frame with the specified title.
+setSize(width: int, height: int): void	Sets the size of the frame.
+setLocation(x: int, y: int): void	Sets the upper-left-corner location of the frame.
+setVisible(visible: boolean): void	Sets true to display the frame.
+setDefaultCloseOperation(mode: int): void	Specifies the operation when the frame is closed.
+setLocationRelativeTo(c: Component): void	Sets the location of the frame relative to the specified component. If the component is null, the frame is centered on the screen.
+pack(): void	Automatically sets the frame size to hold the components in the frame.

FIGURE 12.2 `JFrame` is a top-level container to hold GUI components.

Example1: Creating Jframes

```
import javax.swing.*;

public class MyFrame {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Test Frame");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setVisible(true);

    }
}
```



Title bar, Minimize, Maximize, Close btn.

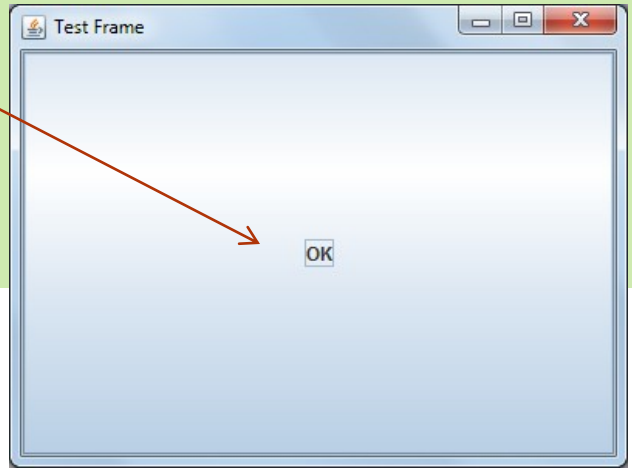
JFrame

Content pane

Resize

Example2: Adding Components to a Frame

```
import javax.swing.*;

public class MyFrame {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Test Frame");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Add a button into the frame
        frame.add(new JButton("OK"));

        frame.setVisible(true);
    }
}
```



JFrame Class

javax.swing.JFrame	
+JFrame()	Creates a default frame with no title.
+JFrame(title: String)	Creates a frame with the specified title.
+setSize(width: int, height: int): void	Sets the size of the frame.
+setLocation(x: int, y: int): void	Sets the upper-left-corner location of the frame.
+setVisible(visible: boolean): void	Sets true to display the frame.
+setDefaultCloseOperation(mode: int): void	Specifies the operation when the frame is closed.
+setLocationRelativeTo(c: Component): void	Sets the location of the frame relative to the specified component. If the component is null, the frame is centered on the screen.
+pack(): void	Automatically sets the frame size to hold the components in the frame.

FIGURE 12.2 `JFrame` is a top-level container to hold GUI components.

Layout Managers

- UI components are placed in containers.
- Each container has a layout manager to arrange the UI components within the container.
- Layout managers are set in containers using the `setLayout(LayoutManager)` method in a container.
- Some basic `LayoutManager` types are:
  - `FlowLayout`,
  - `GridLayout`,
  - `BorderLayout`,
  - Others …

The FlowLayout Class

The `get` and `set` methods for these data fields are provided in the class, but omitted in the UML diagram for brevity.

java.awt.FlowLayout

```
-alignment: int
-hgap: int
-vgap: int
```

```
+FlowLayout()
+FlowLayout(alignment: int)
+FlowLayout(alignment: int, hgap:
   int, vgap: int)
```

The alignment of this layout manager (default: CENTER).

The horizontal gap of this layout manager (default: 5 pixels).

The vertical gap of this layout manager (default: 5 pixels).

Creates a default `FlowLayout` manager.

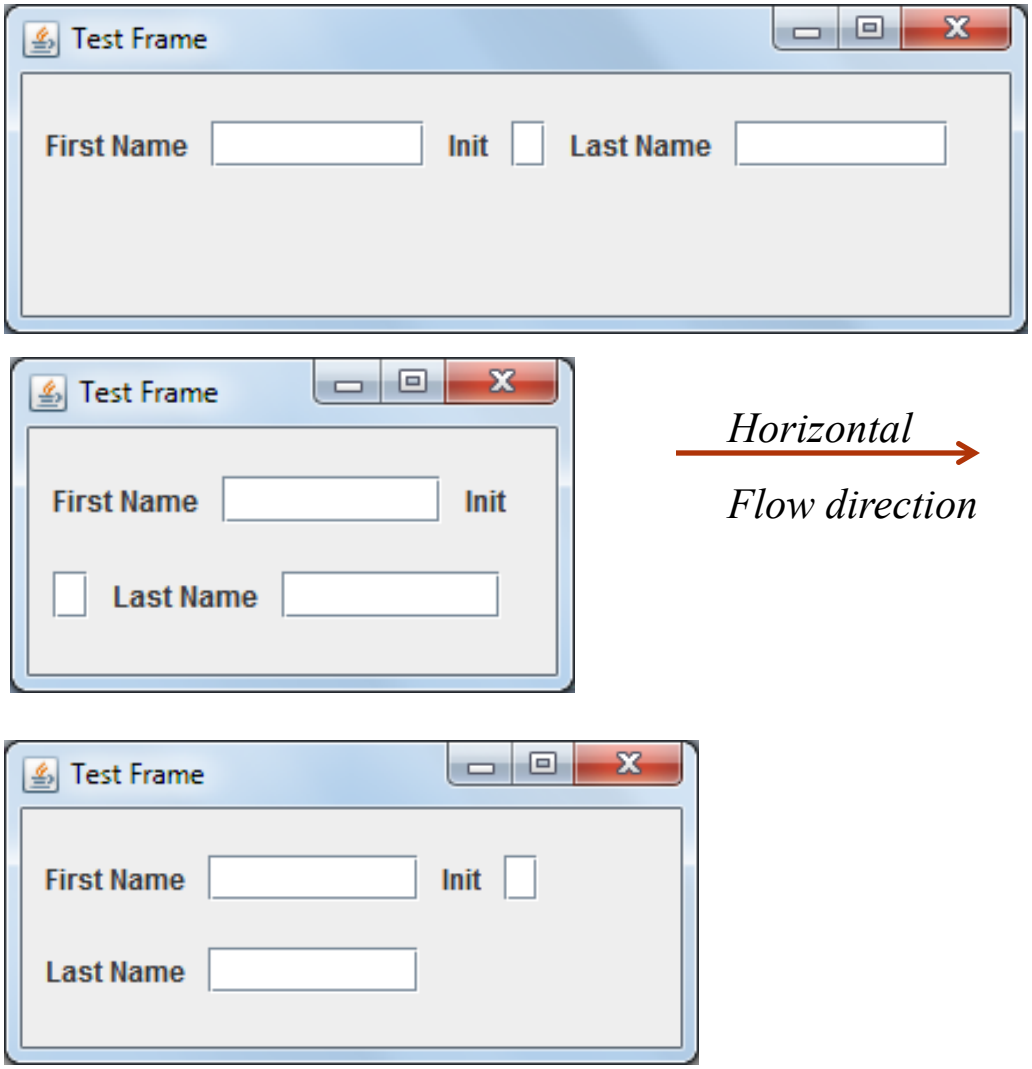
Creates a `FlowLayout` manager with a specified alignment.

Creates a `FlowLayout` manager with a specified alignment, horizontal gap, and vertical gap.

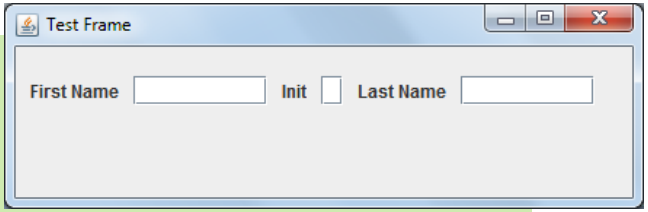
FIGURE 12.4 `FlowLayout` lays out components row by row.

Example3: FlowLayout

This program adds three **labels** and a **text fields** into the content pane of a frame with a (horizontal) `FlowLayout` manager.



Example3: FlowLayout




```java
import java.awt.FlowLayout;
import javax.swing.*;

public class MyFrame {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Test Frame");
        frame.setSize(400, 300);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setLayout(new FlowLayout(FlowLayout.LEFT, 10, 20));

        // Add components to the frame
        frame.add(new JLabel("First Name"));
        frame.add(new JTextField(8));
        frame.add(new JLabel("Init"));
        frame.add(new JTextField(1));
        frame.add(new JLabel("Last Name"));
        frame.add(new JTextField(8));

    }
}
```

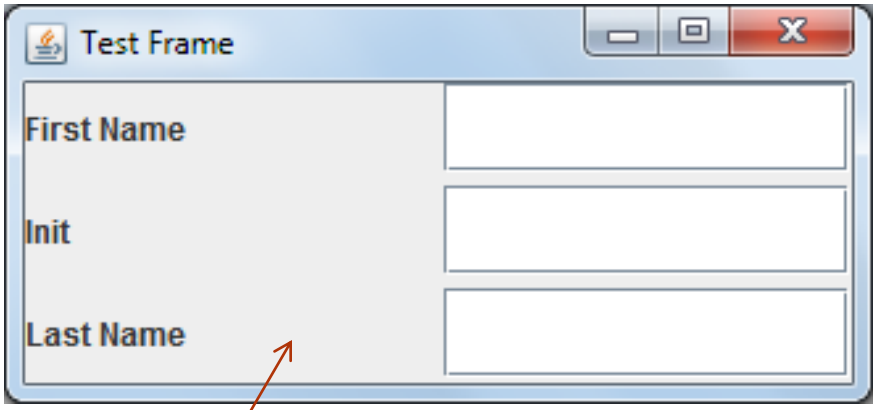
1

2

3

Example4: GridLayout

This program uses a `GridLayout` manager (instead of a FlowLayout manager) to display the labels and text fields.



3 x 2

The GridLayout Class

The `get` and `set` methods for these data fields are provided in the class, but omitted in the UML diagram for brevity.

java.awt.GridLayout

```
-rows: int
-columns: int
-hgap: int
-vgap: int
```

```
+GridLayout()
+GridLayout(rows: int, columns: int)
+GridLayout(rows: int, columns: int,
  hgap: int, vgap: int)
```

The number of rows in this layout manager (default: 1).
The number of columns in this layout manager (default: 1).
The horizontal gap of this layout manager (default: 0).
The vertical gap of this layout manager (default: 0).

Creates a default `GridLayout` manager.
Creates a `GridLayout` with a specified number of rows and columns.
Creates a `GridLayout` manager with a specified number of rows and columns, horizontal gap, and vertical gap.

FIGURE 12.6 `GridLayout` lays out components in equal-sized cells on a grid.

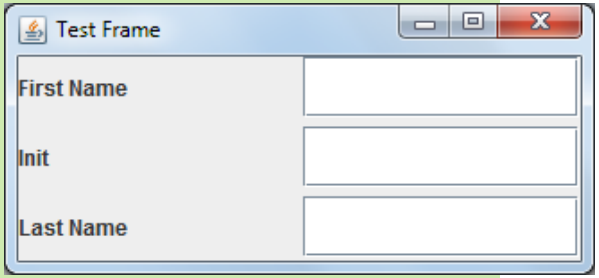
Example4: GridLayout

```
public class MyFrame {
public static void main(String[] args) {
    JFrame frame = new JFrame("Test Frame");
    frame.setSize(400, 300);
    frame.setVisible(true);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    frame.setLayout(new GridLayout(3, 2, 5, 5));

    // Add components to the frame
    frame.add(new JLabel("First Name"));
    frame.add(new JTextField(8));
    frame.add(new JLabel("Init"));
    frame.add(new JTextField(1));
    frame.add(new JLabel("Last Name"));
    frame.add(new JTextField(8));
    }
}
```

1

2

3



The BorderLayout Manager

The `BorderLayout` manager divides the container into five areas:

East, South, West,

North, Center.

Components are added to a `BorderLayout` by using the add method.

```
add(Component, constraint),
      where constraint is:

 BorderLayout.EAST,
 BorderLayout.SOUTH,
 BorderLayout.WEST,
 BorderLayout.NORTH, or
 BorderLayout.CENTER.
```

The BorderLayout Manager

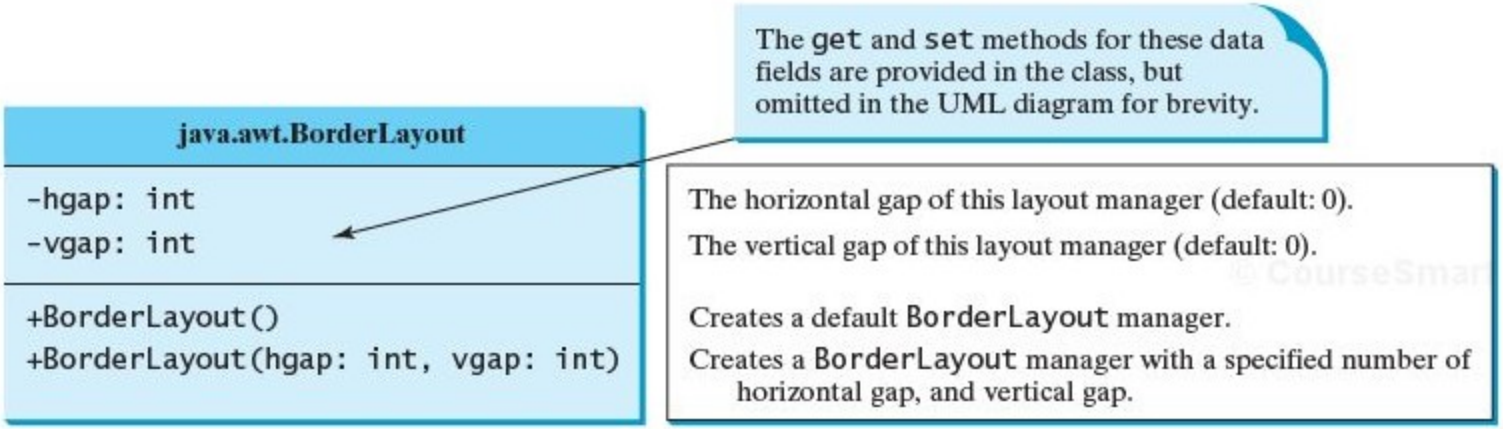
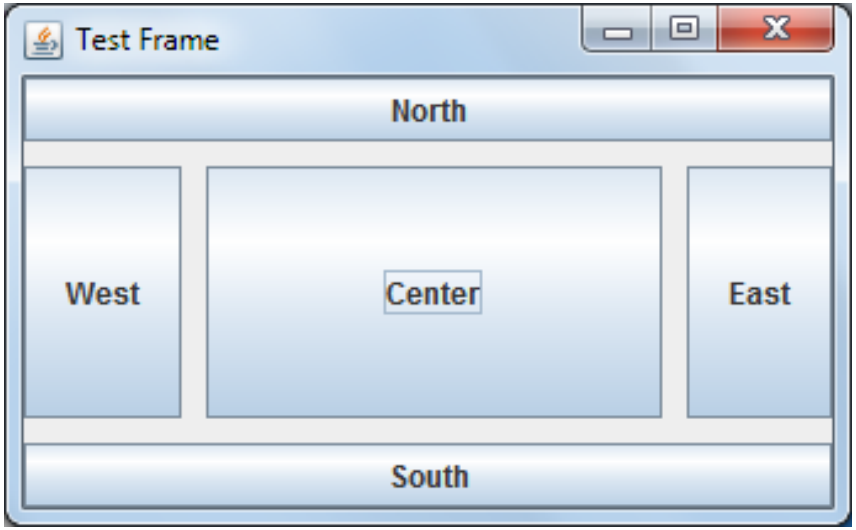


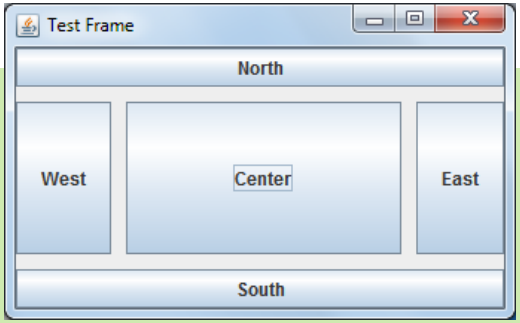
FIGURE 12.8 BorderLayout lays out components in five areas.

Example5: BorderLayout Manager

This version places a JButton in each region of a `BorderLayout`



Example5: BorderLayout Manager




```
public class MyFrame {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Test Frame");
        frame.setSize(400, 300);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setLayout(new BorderLayout(10, 10));

        // Add components to the frame
        frame.add(new JButton("North"), BorderLayout.NORTH);
        frame.add(new JButton("South"), BorderLayout.SOUTH);
        frame.add(new JButton("Center"), BorderLayout.CENTER);
        frame.add(new JButton("East"),  BorderLayout.EAST);
        frame.add(new JButton("West"),  BorderLayout.WEST);
    }

}
```

1

2

3

The Color Class

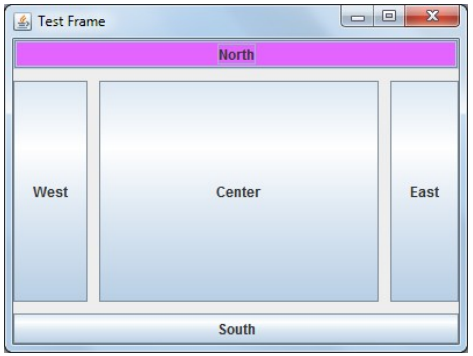
RGB Colors are made of *red*, *green*, and *blue* components, each intensity is represented by a byte value

- 0 (darkest shade)
- 255 (lightest shade).

Example:

Red Green Blue

```
Color c = new Color(228, 100, 255); //light purple
```



Standard Colors

A number of standard colors are defined as constants in java.awt.Color.

You use then as: `Color.xxx` where xxx is:

```
BLACK,          GREEN,          RED,
BLUE,           LIGHT_GRAY,     WHITE, and
CYAN,           MAGENTA,        YELLOW.
DARK_GRAY,      ORANGE,
GRAY,           PINK,
```

Setting Colors

You can use the following methods to set the component's background and foreground colors:

```
setBackground(Color c)
setForeground(Color c)
```

Example:
The button jBtn shows red text on a yellow background

```
jBtn.setBackground(Color.YELLOW);
jBtn.setForeground(Color.RED);
```

The Font Class

Font Names

Supported in all platforms:

SansSerif, Serif, Monospaced, Dialog, DialogInput.

Font.PLAIN (0), Font.BOLD (1), Font.ITALIC (2), Font.BOLD + Font.ITALIC (3)

```
Font myFont = new Font(name, style, size);
```

Example:

```
Font myFont1 = new Font("SansSerif", Font.BOLD, 16);

Font myFont2 = new Font("Serif", Font.BOLD+Font.ITALIC, 12);

JButton jbtOK = new JButton("OK“);

jbtOK.setFont(myFont2);
```

Finding All Available Font Names

```
GraphicsEnvironment e = GraphicsEnvironment
                              .getLocalGraphicsEnvironment();

String[] fontnames = e.getAvailableFontFamilyNames();

for (int i = 0; i < fontnames.length; i++)
            System.out.println(fontnames[i]);
```

```
Agency FB
Aharoni
Algerian
Andalus
Angsana New
AngsanaUPC
Aparajita
Arabic Typesetting
Arial
Arial Black
Arial Narrow
Arial Rounded MT Bold
Arial Unicode MS
Baskerville Old Face
```

...

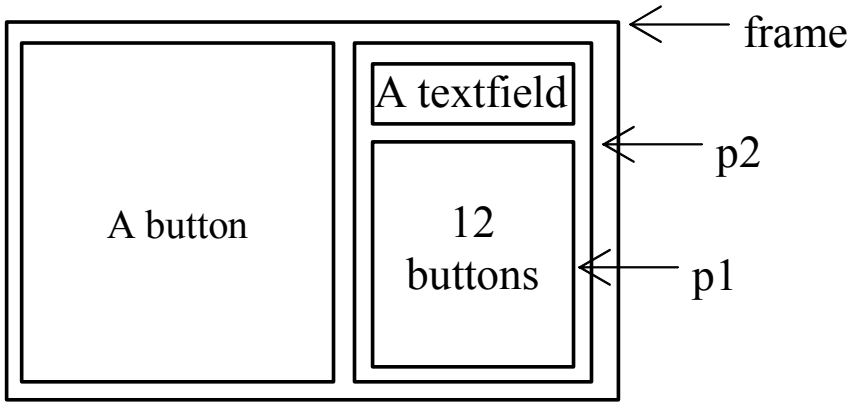
```
Batang
BatangChe
Bauhaus 93
Bell MT
Berlin Sans FB
Berlin Sans FB Demi
Bernard MT Condensed
Blackadder ITC
Wingdings 3
ZWAdobeF
```

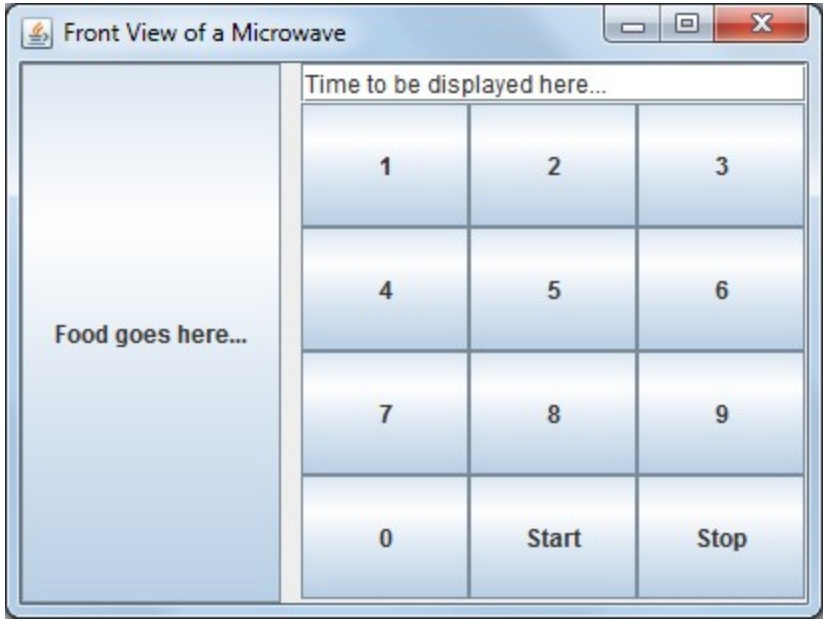
Using **Panels** as Sub-Containers

- `Panels` act as sub-containers for grouping user interface components.
- It is recommended that you place the user interface components in panels and place the panels in a frame.
- You can also place panels in a panel.
- To add a component to JFrame, you actually add it to the content pane of JFrame.
- To add a component to a panel, you add it directly to the panel using the add method.

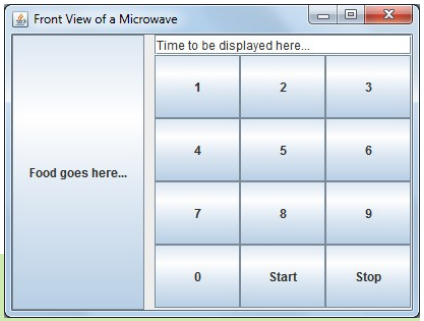
Example6: Testing Panels

- This example uses panels to organize components.
- The program creates a user interface for a Microwave oven.



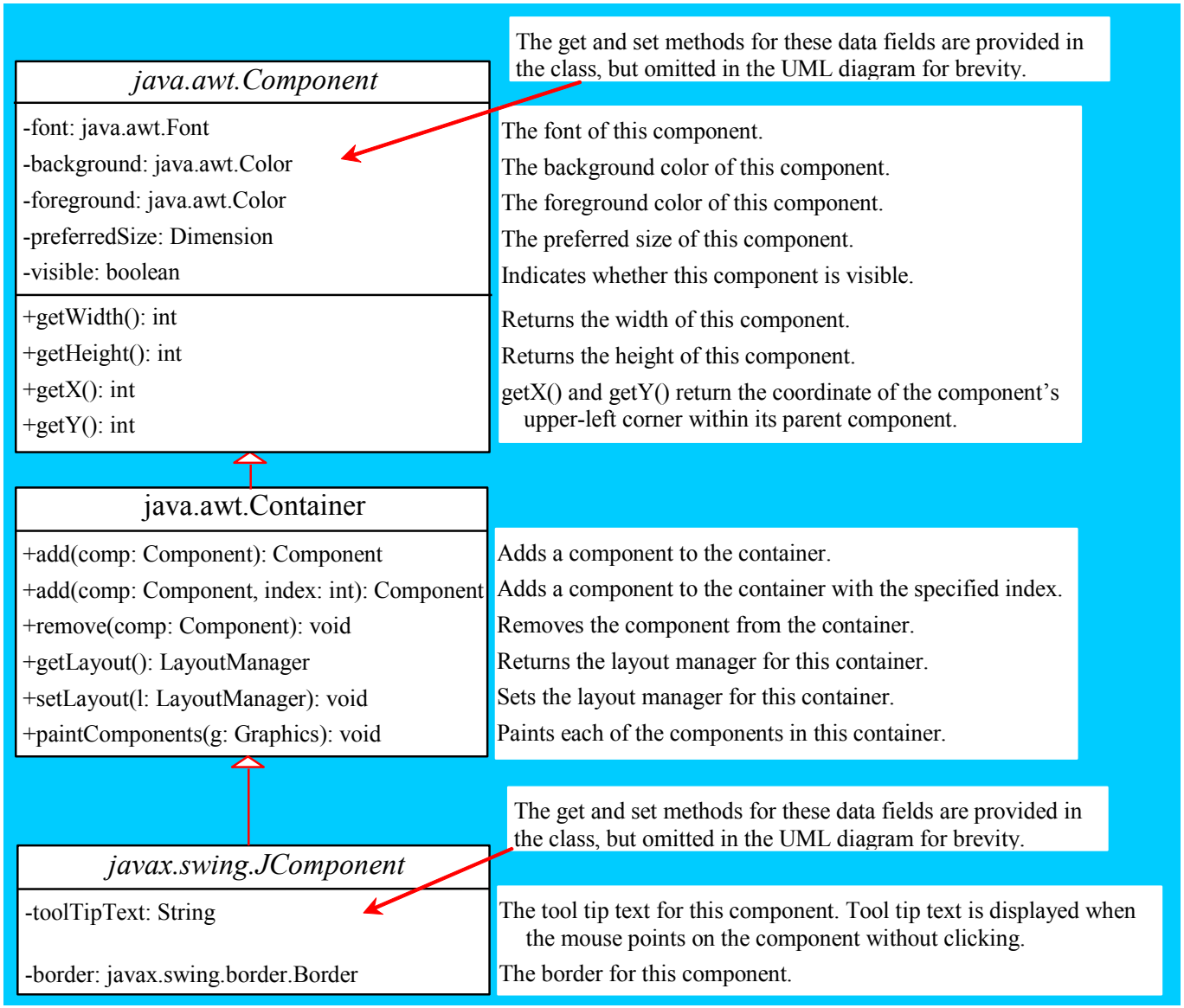


Example6: Testing Panels




```java
public class MyFrame {
    public static void main(String[] args) {
        JFrame frame = new JFrame(
                        "Front View of a Microwave");
        frame.setSize(400, 300);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(
                            JFrame.EXIT_ON_CLOSE);

        JPanel p1 = new JPanel();
        p1.setLayout(new GridLayout(4, 3));

        for (int i=1; i<=9; i++){
            p1.add(new JButton(""+ i));
        }

        p1.add(new JButton("0"));
        p1.add(new JButton("Start"));
        p1.add(new JButton("Stop"));

        //make JPanel p2 to hold a textField and p1
        JPanel p2 = new JPanel(new BorderLayout());

        p2.add(new JTextField(
                "Time to be displayed here..."),
                BorderLayout.NORTH);
        p2.add(p1, BorderLayout.CENTER);

        frame.setLayout(new BorderLayout(10, 10));

        // Add components to the frame
        frame.add(new JButton("Food goes here..."),
                    BorderLayout.WEST);

        frame.add(p2, BorderLayout.CENTER);
    }
}
```

Common Features of Swing Components



Borders

- You can set a border on any object of the JComponent class.
- To create a titled border, use
  `new TitledBorder(String title)`
- To create a line border, use
  `new LineBorder(Color color, int width)`
  where *width* specifies the thickness of the line.

Example: display a titled border on a panel:

```
JPanel panel = new JPanel();
panel.setBorder(new TitleBorder("My Panel"));
```

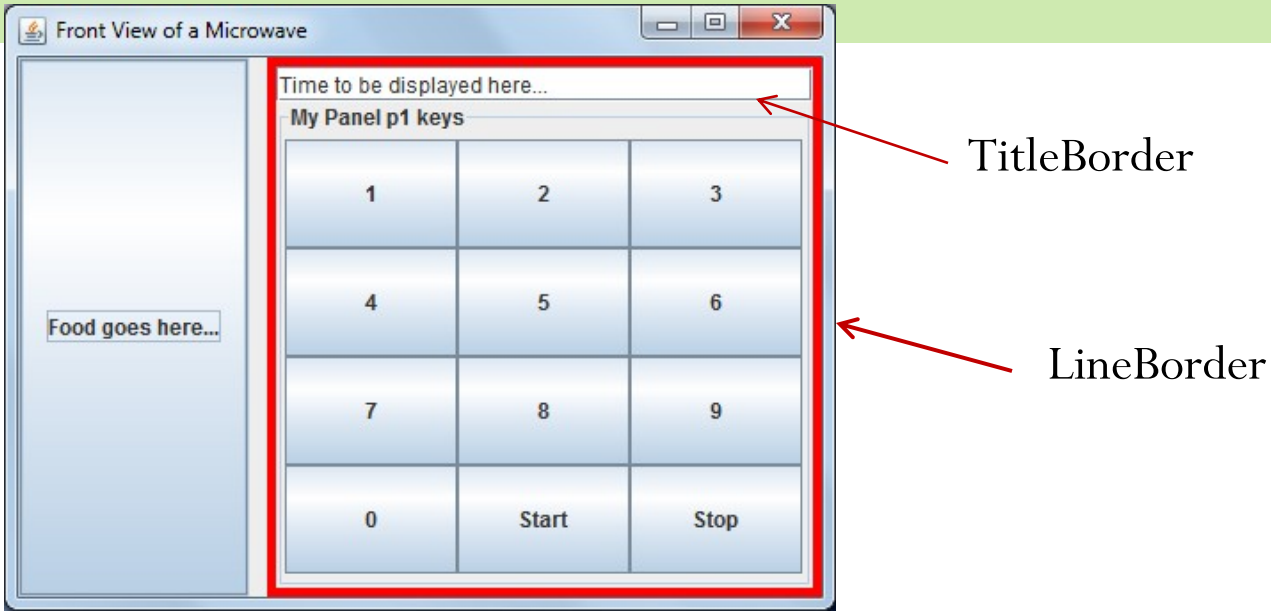
Borders

Example: Modify previous example adding statements

```
p1.setBorder(new TitledBorder("My Panel p1 keys"));
```

and

```
p2.setBorder(new LineBorder(new Color(255,0,0), 5));
```



Test Swing Common Features

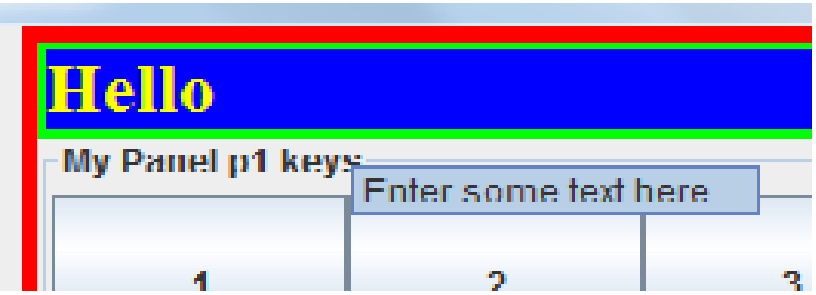
Component Properties

- font
- background
- foreground
- preferredSize
- minimumSize
- maximumSize

JComponent Properties

- toolTipText
- border

Test Swing Common Features




```
JTextField textField = new JTextField("Hello");

textField.setBackground(new Color(0,0,255));      //blue

textField.setForeground(new Color(255,255,0));    //yellow

textField.setFont(new Font("Times New Roman", Font.BOLD, 25));

textField.setBorder(new LineBorder(new Color(0,255,0), 3) );

textField.setToolTipText("Enter some text here ..." );
```

Image Icons

- Java uses the `javax.swing.ImageIcon` class to represent an icon.
- Images are normally stored in image files.

Example:
the following statement creates an icon from an image file *us.gif* in the *image* directory under the current class path:

```
ImageIcon icon = new ImageIcon("image/us.gif");
```

Image Icons

Example: Modify Microwave GUI to add icon

```
ImageIcon myIcon = new ImageIcon("c://temp//Food-128.png");

JButton btnWakeUp =  new JButton("Food here...");

btnWakeUp.setIcon(myIcon);

frame.add(btnWakeUp, BorderLayout.WEST);
```

icon

Splash Screen

A *splash screen* is an image that is displayed while the (slower) application is starting up.

To display a splash screen do this:

```
java -splash:image/us.gf  TestImageIcon
```

displays an image while the program TestImageIcon is being loaded.

Advanced Resources

WindowBuilder

http://code.google.com/javadevtools/wbpro/

http://code.google.com/javadevtools/wbpro/quick_start.html

http://www.java-javafx.com/2011/01/windowbuilder-pro-hello-world-java.html

SWING Builder (Formelry Matisse / NetBeans IDE)

http://netbeans.org/

http://netbeans.org/kb/docs/java/gui-functionality.html#Exercise_1

GWT (Google Web Tool) Plug-in for eclipse

http://code.google.com/webtoolkit/tools/gwtdesigner/tutorials/loginmanager.html

VE (Visual Editor) Archived Eclipse Projets

http://www.eclipse.org/archived/